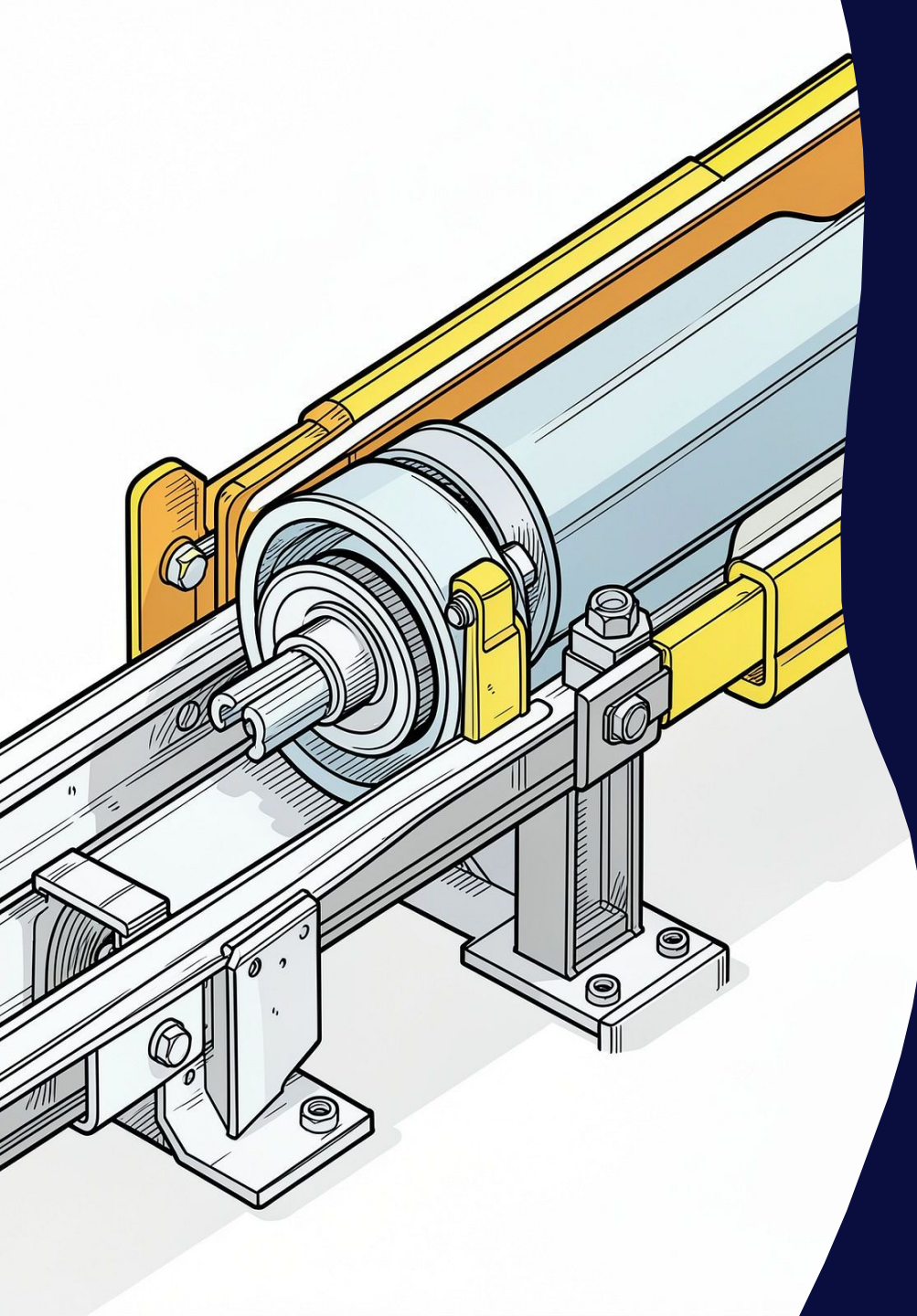


Parametric Adjustable Roller Conveyor — Technical Overview

Automated generation of configurable roller-conveyor assemblies using Python + Autodesk Fusion 360 API. For mechanical/CAD engineers and Fusion automation developers — concise, technical, and implementation-focused.





Project Objective & Core Workflow

Objective

Create parametric conveyor assemblies from user parameters and export validated STEP + BOM.

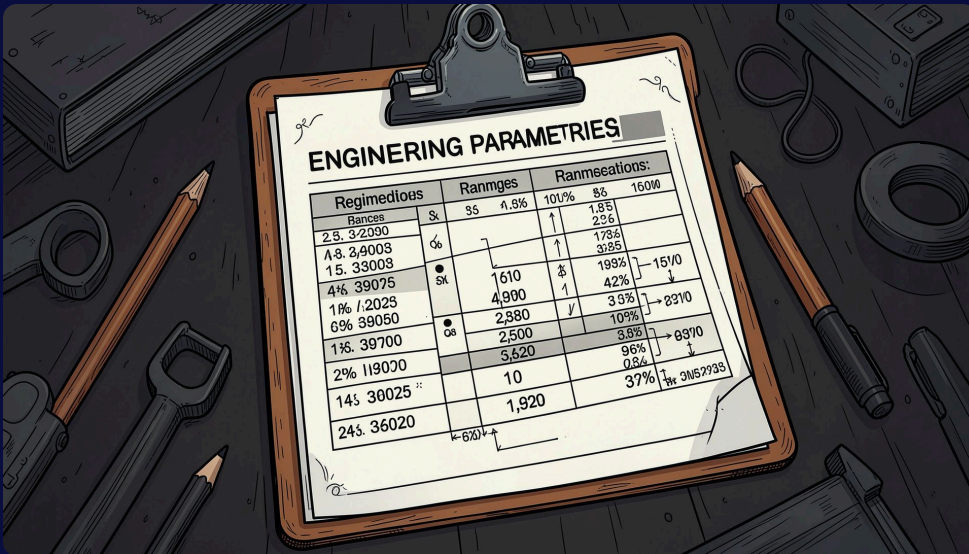
Philosophy

Declare → Compute → Verify → Export. Python declares features; Fusion solves B-rep geometry.

Pipeline

USER INPUT ↓ VALIDATION ↓ PARAMETER SETUP ↓ FEATURE TREE ↓ COMPUTE ALL ↓ VALIDATION ↓ EXPORT

User Inputs & Validation Ranges

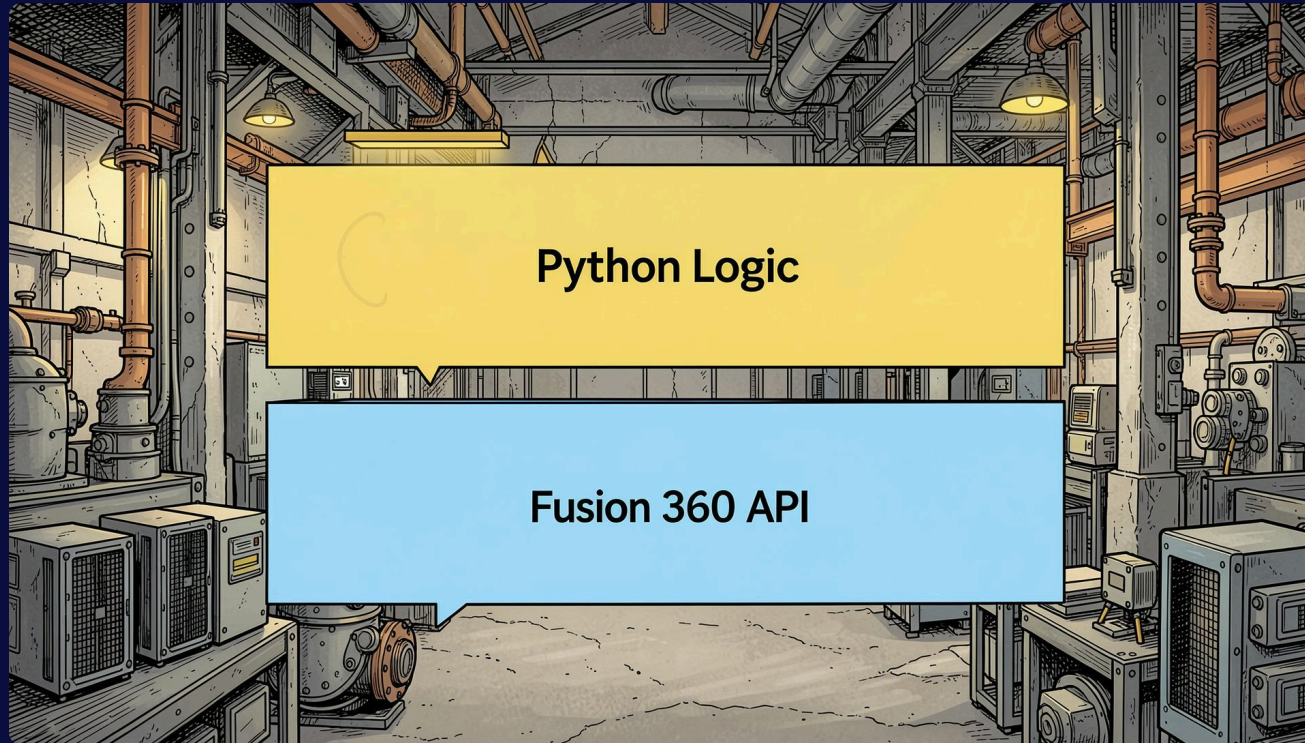


Accepted parameters (units in mm):

- Conveyor Length: 800–2000
- Conveyor Width: 300–600
- Frame Height: 500–900
- Roller Diameter: 40–80
- Roller Spacing: 80–150
- Leg Spacing: 500–1000
- Guard Height: user-defined; Side Guard: ON/OFF

❏ Input validation gates configurations before any Fusion model changes — prevents invalid geometry and export failures.

Software Architecture — Two Layers



Layer 1 — Python Logic

Input parsing, validation, engineering calculations, parameter management, BOM prep, validation logic, report generation.



Layer 2 — Fusion 360 API

Sketches, construction planes, extrusions, patterns, parametric recompute, B-rep creation, STEP export.

Interface: `adsk.core` + `adsk.fusion`

Fusion API Object Chain (Visual Pipeline)



Key objects to interact with: `adsk.core.Application`, `adsk.core.UserInterface`, `adsk.fusion.Design`, `Component`, `Sketch`, `ExtrudeFeature`, `RectangularPatternFeature`, `UserParameter`, `ExportManager`.

Parametric Backbone & Derived Formulas



Primary Fusion UserParameters:

- ConvLength, ConvWidth, FrameHeight, RollerDia, RollerSpacing, LegSpacing, GuardHeight
- Auxiliary: RailW, RailH, LegSide, RollerClearance, GuardThick

Derived formulas (Python & Fusion must agree):

1. $\text{RollerMargin} = \text{RollerDia} / 2 + 10 \text{ mm}$
2. $\text{RollerCount} = \text{floor}((\text{ConvLength} - 2 \times \text{RollerMargin}) / \text{RollerSpacing}) + 1$
3. $\text{LegCount} = \text{floor}((\text{ConvLength} - \text{LegSide}) / \text{LegSpacing}) + 1$

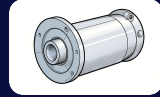
Use `ValueInput.createByString("Expression")` to keep expressions live in Fusion parameters.

Conveyor Generation — Four Stages



Stage A — Rails

Rail sketch dimensioned from ConvLength/ConvWidth. Extrude depth = RailH parameter.



Stage B — Rollers

Master roller uses RollerDia; axial length = ConvWidth - 2×(RailW + RollerClearance). Patterned by RollerCount with RollerSpacing (spacing type: centre-to-centre).



Stage C — Support Legs

Master leg pair extrusion height = FrameHeight - RailH. Patterned by LegCount with LegSpacing.



Stage D — Side Guards

Guards extruded to GuardHeight. When disabled: isSuppressed = true (no zero-height extrusions).

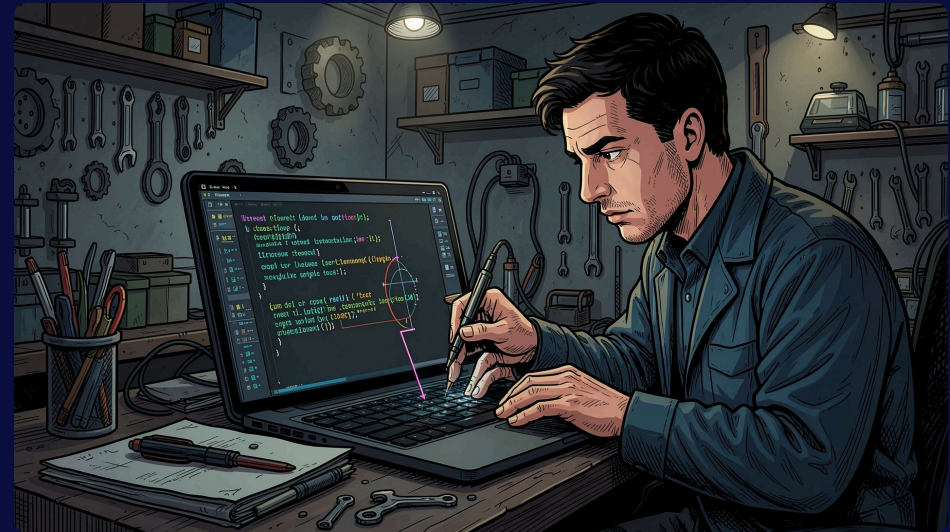
Unit Handling & Common API Pitfalls

Unit rules to avoid 10× errors:

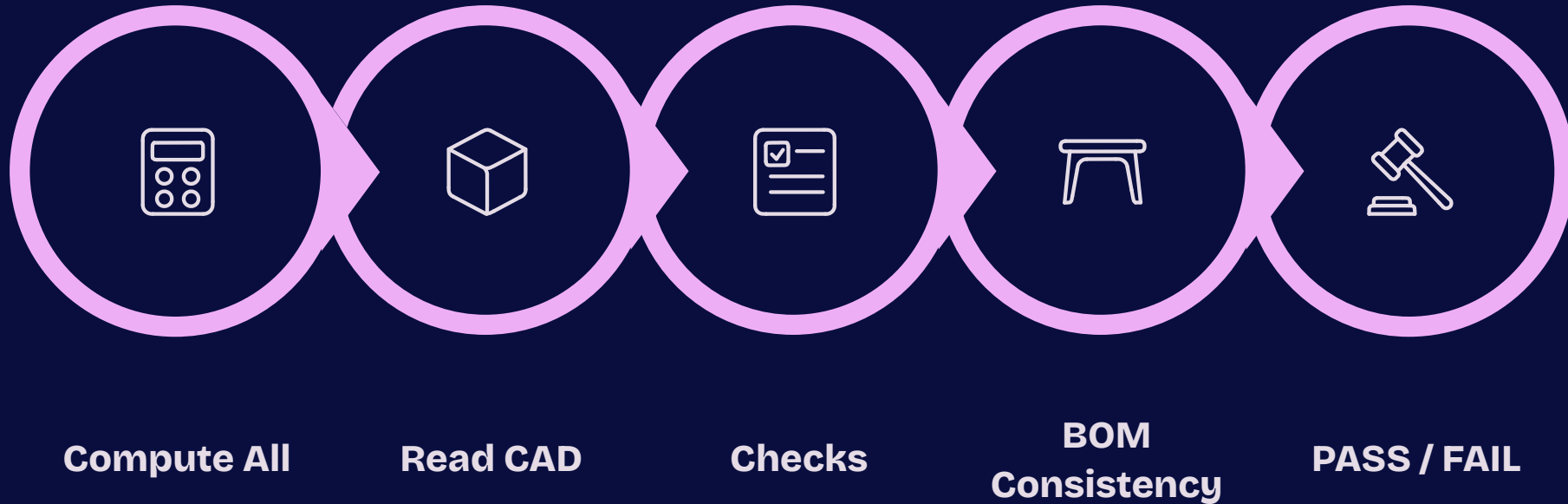
- Design spec: millimetres. Fusion DB units: centimetres.
- `ValueInput.createByString("1400 mm")` → correct unit conversion.
- `Point3D.create(x,y,z)` expects Fusion DB units (cm) — convert beforehand.
- `component.boundingBox` returns cm — convert to mm for validation.

Common Errors & Fixes

- NO ACTIVE DESIGN → require Model document open
- DIRECT MODE → switch to Parametric (timeline required)
- PARAMETER DOES NOT UPDATE → use `createByString` for expressions
- ZERO-HEIGHT GUARD → floor GuardHeight at 1 mm; use suppression



Validation, BOM & Export Workflow



Key points:

- `design.computeAll()` is always called before reading CAD properties.
- Python independently computes expected counts and compares to Fusion parameters (PASS/FAIL).
- BOM quantities originate from the Fusion model to prevent drift.
- Only PASSED configurations are exported as STEP; otherwise export skipped.

Implementation Outline, Testing & Next Steps

Code Modules

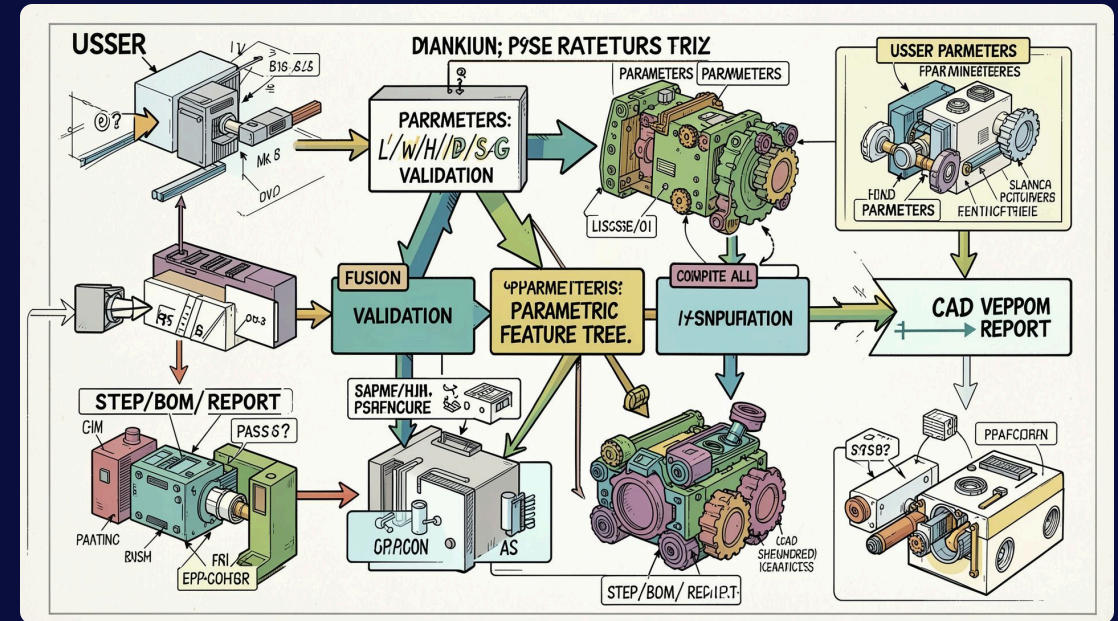
1) input/validation 2) pure-Python engineering logic 3) parameter management 4) CAD generation 5) validation/export 6) Fusion run()

Testing Strategy

Unit tests for pure-Python logic, mocked adsk tests for API behavior, final validation in real Fusion for B-rep and exports.

Future Roadmap

Convert script → add-in with persistent UI, live preview, interference checks, automated optimization, and configuration snapshots.



Python defines intent. Fusion solves geometry. Validation verifies results. Exports are only produced when CAD passes checks.